

聚焦转换器：语境缩放对比训练

Szymon Tworkowski^{1,3*} Konrad Staniszewski^{(1)(3)*}

Mikołaj Pacek^{1(3)*}

Yuhuai

Wu^{6†} Henryk Michalewski^{(3),4} Piotr Miłoś⁽¹⁾⁽²⁾⁽³⁾⁽⁵⁾

¹IDEAS NCBR

²波兰科学院数学研究所

³华沙大学 ⁴谷歌 DeepMind

⁵deepsense.ai

⁶xAI

摘要

大型语言模型具有以上下文方式纳入新信息的卓越能力。然而，由于有效上下文长度的限制，这种方法的全部潜力往往受到制约。解决这一问题的方法之一是赋予注意力层访问由（键、值）对组成的额外上下文的权限。然而，随着文档数量的增加，相关键与不相关键的比例会下降，从而导致模型更加关注不相关键。我们发现了一个被称为 *分心问题* 的重大挑战，即与不同语义值相关的键可能会过度重叠，从而难以区分。为了解决这个问题，我们引入了 *聚焦转换器*（FOT），这是一种受对比学习启发而采用训练过程的技术。这种新颖的方法增强了（键、值）空间的结构，从而扩展了上下文的长度。我们的方法允许对已有的大规模模型进行微调，以延长其有效上下文。我们对 3B 和 7B OpenLLaMA 检查点的微调就证明了这一点。由此产生的模型被我们命名为 LONGLLAMA²，在需要较长上下文的任务中表现出了先进性。我们还进一步说明，我们的 LONGLLAMA 模型能够熟练地管理 256k 上下文长度的密钥检索。

1 简介

语言模型是多个领域取得重大进展的催化剂，包括自然语言处理[Radford等人，2019年；Brown等人，Chen等，2020年]、代码生成[[Polu人LiLewkowycz等人，2021年；等人，20222022；2022年；年年2020年]、定量推理]2023年]。和定理证明和Sutskever，Jiang等人，Mikuła等人，语言模型的核心挑战之一是有效地纳入大量新知识。对模型进行微调的常见做法不仅资源密集、管理复杂，而且并不总是能明确指出如何纳入新知识。例如，对“爱丽丝梦游仙境”这样的文本进行微调，并不能使模型回答有关故事本身的问题，而是

*等效贡献 [†]在 Google Research 工作期间完成的工作。

² 我们发布了 LONGLLAMA  的检查点和源代码，也请参见我们的 colabs。

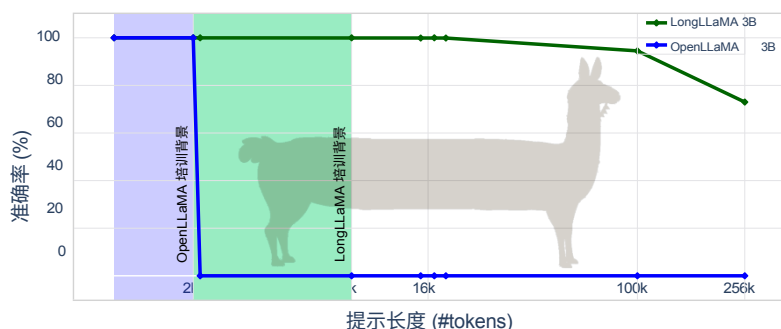


图 1：与原始 OpenLlama 模型相比，LONGLLAMA 3B 在密钥检索方面的准确率。我们的方法可以超越训练长度，在上下文长度为 100k 时达到 94.5% 的准确率，在 256k 个 token 时达到 73% 的准确率，而基线则无法处理超过其训练长度（2k）的上下文。

而基线则无法处理超过其训练长度（2k）的上下文。一种很有前途的替代方法是将新知识整合到上下文中，这种方法不需要训练，但在很大程度上受到模型有效上下文长度的限制。为了使这种方法适用于大型知识数据库（如大型代码库），模型需要管理的上下文长度可扩展至数百万个标记。

在这项研究中，我们强调了增加上下文长度的主要障碍之一：随着文档数量的增加，相关与不相关的关键字的比例会越来越小。标准的训练程序经常会导致与无关值相关的键和与相关值相关的键之间出现重叠，从而加剧了模型区分它们的任务。我们将这一挑战称为**分心问题**。

我们提出了**聚焦转换器** (FOT)，这是专门为解决这一问题而开发的创新技术。聚焦转换器允许注意力层的一个子集通过 k 近邻 (kNN) 算法访问额外的（键、值）对上下文，类似于[Wu 等人，2022]。中使用的方法这种机制有效地延长了上下文的总长度。Focused Transformer 的独特之处在于其借鉴对比学习的训练程序。这种方法解决了注意力分散的问题，有助于扩大语境容量。具体来说，在训练阶段，我们会故意让所选的注意力层子集接触到相关和不相关的关键字（如来自无关文档的负面样本）。这种策略可以激励模型区分与不同语义值相关的关键字，从而增强其结构。

我们介绍并提供了带有 FOT 的微调 OpenLlama 模型 LONGLLAMAs (🦙)，证明我们的方法在训练过程中不需要长语境，并且可以应用于现有模型。值得注意的是，LONGLLAMAs 在需要长语境建模的任务中表现出了显著的改进。特别是，它们可以在密钥检索任务中管理 256k 的上下文长度[Mohtashami 和 Jaggi，2023]。

我们的研究贡献如下：

1. 我们指出分心问题是一个重大挑战，也是在 Transformer 模型中扩展上下文长度的主要障碍，尤其是在多文档场景中。
2. 我们开发了聚焦变换器（FOT），旨在缓解分心问题。FOT 包括一个独特的训练目标，可改进（键、值）结构，从而能够使用大量附加上下文和 k 近邻查找来扩展上下文长度。
3. 我们的方法简单易用，而且可以在不修改架构的情况下扩展模型上下文，并通过经济有效的微调来实现。我们在 3B 和 7B OpenLlama 检查点上进行了演示。由此产生的模型被命名为 LONGLLAMAs，在一些任务中显示出了增强效果，这些任务得益于在扩展上下文中增加了少量演示，例如 TREC [Li and Roth, 2002, Hovy et al., 2001] 和 WebQS [Berant et al., 2016]。

2013]我们还证明,对于密钥检索 Mohtashami 和 Jaggi [2023],我们的 LONGLLAMA 模型能成功处理 256k 的上下文长度。

4. 我们进一步研究了 FOT 在不同数据集和模型规模下的能力。我们发现,在一个基准词典查询任务中,用 512 个词组的总上下文训练出来的 FOT 可以推断出 1600 万个词组。我们还在书籍 (PG-19)、数学 (arXiv)、代码 (GitHub) 和形式化证明 (Isabelle) 等长语境语言建模任务中对 FOT 进行了评估,结果表明它的易错性比基线有所提高。

2 相关工作

长上下文变换器架构 为增加变换器的上下文长度,人们开发了多种方法,主要侧重于减轻注意力计算的二次复杂性。例如,Transformer-XL [Dai 等人, 2019] 缓存了之前的上下文,并实现了上下文随层数的线性扩展。Longformer [Beltagy 等人, 2020] 采用了一种注意力机制,允许标记稀疏地关注远处的标记,从而降低了计算复杂度。BigBird [Zaheer 等人, 2020]、LongT5 [Guo 等人, 2021] 和 [Dao 等人, 2022] 也使用稀疏注意力来处理长序列。Kaddour 等人, 2023] 对不同的效率考虑进行了研究,结果表明它们的收益有限。分层变换器[Nawrot 等人, 2021 年, 2023 年]对中间层的激活进行下采样,以减少计算量并实现更长的上下文。COLT5[Ainslie等人, 2023]提出了条件计算,以节省内存并实现更长的上下文。Memorizing Transformer [Wu 等人, 2022 年] 使用 kNN 查找来拾取最相关的词块,这也可以看作是降低注意力计算复杂度的一种方法。我们的工作遵循了这一方法,旨在训练一个能处理更长注意上下文长度(例如,通过减轻分心问题)的密钥空间,从而具有更好的长上下文能力。

微调 LLMs 以延长检索时间 之前的 RETRO [Borgeaud 等人, 2022 年] (RETROfitting) 和 Memorizing Transformer [Wu 等人, 2022 年] 等研究已经证明,微调现有 LMs 以增加新功能而无需重新训练整个模型,是一条大有可为的道路。与这些方法不同的是,我们的方法并不以检索为目的,而是以扩展模型上下文为目的。与 RETRO 不同的是,我们提出了一种单阶段的语境扩展方法,而不是检索--然后--嵌入的两阶段方法。我们在附录 C.3。中提供了与 Memorizing Transformer 的更详细比较最近,许多研究都在探索对 LLaMA 进行微调,以扩展其上下文长度。Landmark attention [Mohtashami and Jaggi, 2023] 提出了一种将 LLM 上下文压缩为地标的方案,将 LLaMA-7B 的上下文长度增加到 32K。位置插值 (PI, [Chen 等人, 年2023] 和 [kaiokendev, 2023 年]) 对旋转位置编码方案进行了修改,从而可以对 32K 上下文进行微调。与这项工作不同的是,我们的方法并不依赖位置编码,这与 [Haviv 等人, 2022] 的。研究结果一致在额外上下文中移除位置编码可以让我们推断出 256k 的词块,尽管模型只在高达 8K 的序列上进行过训练,理论上上下文长度是无限限制的。

零点法 KNN-LM [Khandelwal 等人, 2019] 表明,通过结合两种概率分布,可以提高 LLM 的性能。一个是由预训练模型创建的,另一个是基于当前处理 token 的嵌入与从大型数据库中检索到的 token 的嵌入之间的相似性。同时,我们在注意力层的一个子集中扩展了模型上下文,从而有可能在这一扩展上下文中进行推理。大型语言模型的并行上下文窗口[Ratner 等人, 2023 年]介绍了一种无需训练即可扩展语言模型上下文的方法。他们通过并行独立嵌入多个上下文窗口,并只允许一个标记子集关注所有窗口来实现这一目的。另一方面,我们对现有的模型进行了微调,允许所有的标记都关注之前的所有标记,但只在一个子集层中。此外,我们的方法还能改进现有模型的键值空间结构。

对比学习 对比学习旨在通过比较正反两方面的例子来学习良好的表征。CLIP [Radford 等人, 2021 年] 和 SimCLR [Chen 等人, 2020 年] 是两种流行的对比学习方法,在图像领域取得了最先进的性能。在对比预训练过程中,负例子被保留在同一批次中,以学习区分

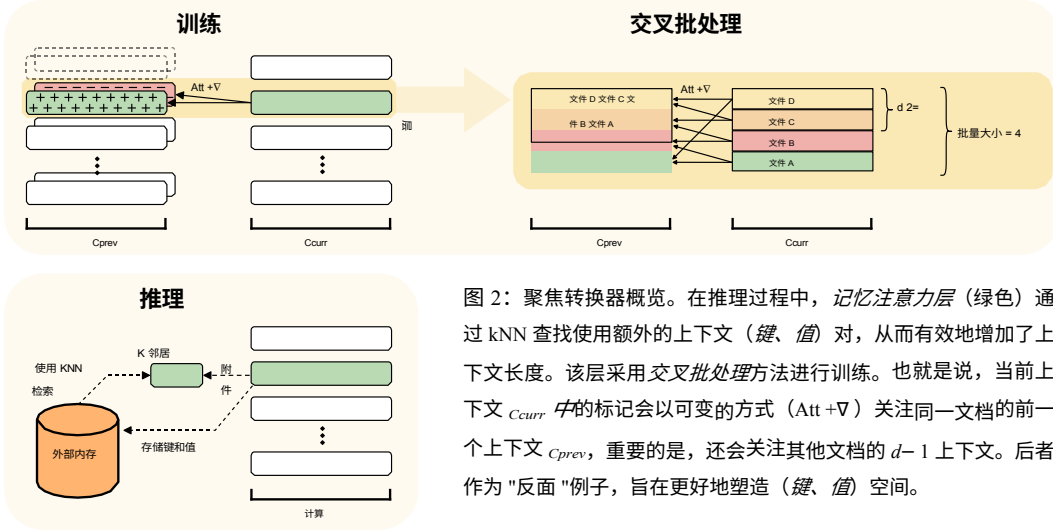


图 2：聚焦转换器概览。在推理过程中，**记忆注意力层**（绿色）通过 kNN 查找使用额外的上下文（**键、值**）对，从而有效地增加了上下文长度。该层采用**交叉批处理**方法进行训练。也就是说，当前上下文 C_{curr} 中的标记会以可变的方式（ $Att + V$ ）关注同一文档的前一个上下文 C_{prev} ，重要的是，还会关注其他文档的 $d-1$ 上下文。后者作为“反面”例子，旨在更好地塑造（**键、值**）空间。

这些表征来自正面例子。正如[Gao 等人, 2021b]。所述，对比学习中批量大小的缩放已被证明能提高表征的质量有人提出[Gao 等人, 2019]，语言建模中的嵌入空间存在退化现象，即嵌入紧紧包裹在一个狭窄的锥体中，很难区分它们。TRIME[Zhong等人, 2022]提出了一种训练方法，专为训练具有记忆增强功能的LM而设计，它使用否定来提高表征的质量。这种方法与我们的方法的主要区别在于，我们将否定词纳入所选的注意层子集，而不是在输出层进行插值，并使用标准的语言建模损失。TRIME[Zhong等人, 2022]也侧重于从大型数据库中进行检索，而我们则侧重于扩展模型的语境。ContraCLM [Jain 等人, 2023] 在训练过程中在标记和序列两个层面上应用对比损失，以促进更均匀分布的各向同性表征。在文本语义相似性基准上，它被证明能提高表征的辨别能力。ContraCLM 专注于提高表征的一般表达能力，而我们的研究则引入了受对比启发的技术，专门用于训练注意力机制，以处理更长的上下文长度。尽管如此，在未来的工作中，探索其他对比学习目标可能会有利于进一步改进关键结构。

3 FOT：聚焦转换器

我们的方法，即聚焦变换器（FOT），是变换器模型的一个简单的即插即用扩展，既可用于训练新模型，也可用于微调现有的、可能是大型的、具有较长上下文的模型。为此，FOT 使用了**记忆注意力层**和**交叉批处理**训练程序。记忆注意力层可以让模型在推理时从额外的上下文中检索信息，从而有效地扩展上下文。交叉批处理训练程序会使模型偏向于学习（**键、值**）表征，而记忆注意力层很容易使用这些表征。FOT 架构概览见图 2，伪代码见附录 L。

3.1 记忆注意力层

在推理过程中，记忆注意力层 L 可以访问额外的上下文。也就是说， $\ell \in L$ 中的每个查询都会关注本地上下文中的前面关键字和内存中最匹配的 k 个关键字（即与查询的内积最大）。内存键根据与查询的内积进行排序，并使用 kNN 搜索算法进行检索。我们使用的是 FAISS [Johnson 等人, 2017]。中实现的精确 kNN 搜索内存由事先经过 ℓ 处理的（**键、值**）对增量填充。我们的内存注意力层设计与[Wu 等人, 2022]密切相关，我们沿用了其大部分设计选择，但门控除外，我们用一种更简单的机制取代了门控，结果证明这种机制在我们的应用中更为有效。详见 C.3 节和附录 B.2。我们删除了内存层中的位置编码

在我们的所有模型中，除了 LONGLLAMA。这使得 LONGLLAMA 检查点可以直接替代 LLaMA 检查点。我们将 kNN 搜索算法视为完全密集注意力的近似值，这为未来的提速打开了大门。

3.2 交叉批处理训练程序

我们的训练程序是一种训练（或微调）基于变换器的架构的新方法，目的是改善（键、值）空间的结构。其主要动机是塑造这一空间，以便记忆注意力层 $\ell \in L$ 可以轻松地关注相关信息。受对比学习（contrastive learning）的启发，关键的想法是让 ℓ 接触到来自给定文档当前和之前本地上下文（正向）的（键、值）对，以及来自不相关文档（负向）的 $d-1$ 上下文。重要的是，这是以可微分的方式实现的。

为此，我们使用了一个数据管道，其中批次的每个元素都对应不同的文档。我们为每个处理过的文档嵌入上一个 (C_{prev}) 和当前 (C_{curr}) 本地上下文。我们的流程概览见图 2。具体来说，对于 C_{curr} 中的每个文档 δ ，我们都会创建一个集合 $\{p^{(\delta)}_{(i)}\}_{i=1}^d = \{(\ell, (1, \dots, d))\}$ ，该集合由来自 δ 上一个本地上下文的（键、值）对（正值）和来自 C_{prev} 的 $d-1$ 个其他上下文的对（负值）组成。我们还尝试改变不同批次元素的前一个上下文和否定词的数量。

这一操作是完全可微分的，因此我们改进了 p^δ 中的所有（键、值）对。其次，这个过程很容易实现；它不需要任何额外的损耗（即使用标准转换器训练目标），只需在数据加载管道和自我注意力的微小变化层面上完成。唯一的新超参数是 d ，它规定了正样本和负样本的比例。通常情况下，我们认为一开始使用较小的 $d \leq 8$ （否则，模型会倾向于忽略之前的局部上下文），之后再改用较大的值，如 $d \geq 64$ 。附录 B.3 提供了有关该方法的更多细节。清单 1 概述了交叉批处理的实现。

3.3 分心问题

在本节中，我们将对所谓的分心问题进行概念化，并假设它是处理长篇多文档上下文（如大型代码库）的关键问题之一。也就是说，在标准的训练过程中，模型无法区分不同文档中的关键字。我们测量的结果是，注意力均匀分布在有关联和无关联的文档上；见图 3。更准确地说，对于文档 δ ，让 w_{ij} 成为与以下内容相关的软最大权重

p^δ 如第 3.2 节所述构建
我们将正向注意力质量定义为
$$r_{(d)}^{(\delta)} := \frac{\sum_j w_{ij}}{\sum_{i=1}^d \sum_j w_{ij}}$$

()
$$r_d \approx 1/d.$$

这可以理解为，注意力同样被正键（来自 $i=1$ 的当前文档）和负键所分散。这是不可取的。

因为在缩放内存时，注意力会越来越分散。我们的研究表明，交叉批处理在很大程度上缓解了注意力分散的问题，从而使注意力更加集中。更多信息请参见附录 B.4。在第 5.3 节中我们还展示了注意力分散问题对复杂度等指标的有害影响。

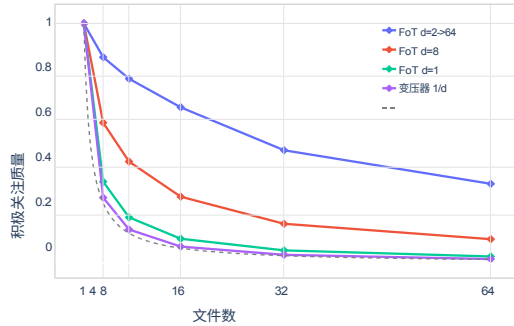


图 3：分心问题。我们将使用不同参数 d 值训练的 FoT 与标准变换器进行比较。

基线。在评估过程中，两个模型都会看到之前的本地上下文和其他文档中的一些上下文。

在交叉分批训练过程中所选择的层。对于文档 δ ，我们根据 p^δ 来衡量注意力的分布情况。标度 x ：模型所能看到的文档上下文数量。标度 y ：当前文档前一个本地上下文平均注意力质量。

4 LONGLLAMA🐘: 利用 FOT 扩展 LLaMA 的上下文长度

我们的工作承诺之一是，FOT 可用于微调现有的大型模型，以扩展其上下文长度。在本节中，我们将证明这一点。我们使用为 *1T* 标记训练的 OpenLLaMA-3B 和 OpenLLaMA-7B 模型作为起点，并使用 FOT 对其进行微调。我们证明，由此产生的模型（我们称之为 LONGLLAMAs）能够超出其训练上下文长度（甚至达到 256K），并保持在短上下文任务中的性能。我们在 GitHub 上发布了推理代码：https://github.com/CStanKonrad/long_llama，并在 Hugging Face 上发布了 LONGLLAMA-3B 检查点：https://huggingface.co/szymon/long_llama_3b。我们注意到，我们的检查点是向后兼容的，即可以与任何现有的 LLaMA 推理代码（包括 Hugging Face 和其他实现）一起使用，尽管没有长语境功能。

4.1 实验设置

模型的结构与 OpenLLaMAs 相同，参见耿和刘[2023]和附录 A.1。我们使用 $L = \{6, 12, 18\}$ （或 $L = \{8, 16, 24\}$ ）作为 3B（或 7B）LONGLLAMA 模型的存储层。我们使用 FOT、8k 上下文长度和基于 RedPajama [TogetherComputer, 2023] 的数据集混合物，在 10B (resp. 3B) 标记上对模型进行微调，见附录 A.3。

与标准的 FOT 程序有三个细微差别。首先，我们保留了内存层本地上下文中的位置编码（这对于 FOT 来说并非必要，但却使我们的检查点与任何现有的 LLaMA 推理代码库完全兼容）。更准确地说，本地上下文中的查询和键值（最多 2K 个标记）采用标准的 LLaMA 旋转位置编码，而内存键值的编码则视同它们在本地上下文窗口中的位置为 0。其次，我们使用密集注意力而不是 kNN 检索，因为我们发现两者的性能差异不大，而且实施起来也更简单。第三，我们修改了交叉批处理训练程序，以便对附加上下文的数量和正负样本的比例进行更精细的控制。所有这些不同之处详见附录 A.2。

4.2 密钥检索任务的上下文长度外推法

我们首先测量的是 LONGLLAMA 的有效上下文长度，即标记之间能够有效地相互关联的距离。我们使用 [Mohtashami and Jaggi, 2023]，这是中介绍的密钥检索一个旨在测量这一特性的合成任务。在这项任务中，模型必须检索随机放置在长提示符中的密码。结果如图 1 所示--重要的是，我们的 3B 模型在解决这项任务时的能力远远超过了它的训练上下文长度 8K，在长度为 100K 的提示符下达到了 94.5% 的准确率，在长度为 256K 的提示符下达到了 73% 的准确率。

4.3 研究论文的问题解答

在表 6 中，我们列出了 SCROLLS [等人Dasigi, Shaham, 等人2022] 中 Qasper [2021] 验证集的性能，并将我们的结果与 LongChat 7B [Ma 和 Zhang, 2023] 以及两个基线短上下文模型进行了比较。我们注意到，我们的模型因语境长度的增加而有所提高。

4.4 用更长的上下文来提高少量学习的准确性

我们在两个下游任务上测量了这些模型的长语境能力，即 TREC 问题分类 [Li and Roth, 2002, Hovy et al.2013]。2001] 和 WebQS 问题解答 [Berant et al., 我们沿用了 [Hao 等人, 2022] 的。实验设置也就是说，在给定的上下文长度内，我们用尽可能多的演示示例来提示模型。我们不像[郝等人, 2022]那样使用结构化提示--而是直接提供上下文中的所有演示。

我们发现，在 TREC 数据集上，较长上下文的准确率有明显提高，在 WebQS 数据集上也有一些改进（见表 1）。TREC 数据集包含 50 个类别。模型的任务是预测给出上下文示例的类别标签。符合标准上下文长度（2K）的例子只有 100 个；对于一个给定的问题来说，没有类别例子的情况并不罕见，这使得任务无法完成。增加上下文长度和示例数量可以降低这种风险。此外，有更多给定类别的示例也可能是有益的。

表 1: LONGLLAMA 的少量上下文学习性能; 在 TREC 和 WebQS 上的准确率。在 TREC 数据集上, 我们看到额外的上下文带来了显著的收益。为了计算结果, 我们从训练集中抽取了 20 次试验, 取平均值来计算语境中的演示; 结果显示, TREC 和 WebQS 的置信区间分别小于 1%和 0.1%。

数据集	TREC		WebQS	
	朗格拉玛 3B	朗格拉玛 7B	朗拉马 3B	朗拉马 7B
2K	67.0	63.2	21.2	25.5
4K	71.6	72.7	21.4	26.4
6K	72.9	74.9	22.2	27.2
8K	73.3	75.9	22.4	27.7

表 2: 在 4K 上下文中进行标准微调 (基线) 和在相同上下文长度下对 1B 标记进行 FoT 微调的 "少量拍摄" 上下文学习性能比较。在 TREC 中, FoT 能够利用训练上下文长度之外的额外示例, 在 8K 上下文长度时获得更高的准确率, 而基线则无法做到这一点, 因为其上下文长度限制在 4K。

数据集	TREC		WebQS	
	基线	FoT (我们的)	基线	FoT (我们的)
2K	52.8	55.6	20.7	20.8
4K	57.2	60.9	18.7	21.0
6K	-	61.7	-	21.2
8K	-	62.5	-	20.7

4.5 与标准长上下文微调的比较

在本节中, 我们将 FOT 与标准长语境微调进行比较, 结果表明, 在微调所使用的语境长度方面, FOT 已经取得了更好的性能, 而且重要的是, 它可以推断出超出这一语境长度的情况, 而基线则不具备这种能力。

为了进行比较, 我们对两个模型进行了微调, 一个是用 FOT 训练的模型, 另一个是用标准微调训练的模型 (基线) (类似于[MosaicML, 2023, Nijkamp 等人, 2023])。在这两种情况下, 我们都使用了在 1B 标记上使用 4K 上下文长度进行微调的 3B 模型。我们在第 4.4。节所描述的环境中, 对这两种模型在一些少量下游任务上的表现进行了评估

在大多数情况下 (见表 2), 当扩展上下文 (从 OpenLLaMA 使用的 2K 到我们的微调中使用的 4K) 提供了更多的少量演示时, 我们观察到准确率有所提高。在 TREC 中, 两个模型都从额外上下文中获得了显著提高, 而在 WebQS 中, 标准微调基线并没有从扩展上下文中获得任何提高。值得注意的是, 使用 FOT 进行微调的模型在使用超出其训练长度的上下文长度 (6K 和 8K) 进行评估时, 准确率进一步提高。这表明 FOT 具有基线所不具备的外推能力 (如图 1)。

4.6 短语境任务中的表现

由于训练数据分布发生了变化, 对更长上下文的微调可能会损害原始上下文长度 (2K) 的性能。我们使用 LM Evaluation Harness 库 [Gao 等人, 2021a]。对 LONGLLAMA 模型进行了评估, 结果表明情况并非如此在大多数任务中, 其性能保持不变; 详情请参见附录 A.4。这也证实了 LONGLLAMAs 可以用来替代 LLaMA 模型, 因为它们与原始的 LLaMA 推理代码兼容。

5 FOT 分析

在本节中, 我们将在较小的模型上进行大量实验, 以分析和进一步验证我们的方法。特别是, 我们回答了以下问

题：(1) 在推理时缩放上下文长度时，FOT 的表现如何？(2) FOT 能否用于扩展现有预训练模型的上下文长度？

(2) FOT 能否用于扩展现有预训练模型的上下文长度？(3) FOT 能多有效地处理分心，以及这种能力如何在长语境语言建模任务中转化为更高的性能？此外，我们还对我们的方法进行了消融研究和补充分析。

5.1 实验设置

架构 在本节描述的实验中，我们使用纯解码器的 Transformer [Vaswani 等人, 2017] 模型，共有 12 层和 1.84 亿个参数（除非另有说明）。根据 Wu 等人的研究[2022]，我们选择 $\ell=8$ 作为记忆注意层。我们调整了 $k=128$ ，即 kNN 检索到的顶级密钥数。在大多数实验中，我们从较小的交叉批次维度 $d \leq 8$ 开始训练，经过一段时间的训练后，再切换到 $d \geq 64$ 。有关架构和超参数的更多详情，请参阅附录 B 和附录 E。

评估 我们区分了两种评估环境：单文档（简称单文档）和多文档（简称多文档）。单文档设置通常用于评估处理长上下文的模型。在这里，我们会为每个新文档清除内存，确保上下文中只有当前文档可用。多文档设置会保留多个文档的内存，而不会重置。这种情况测试的是模型是否能忽略无关信息并专注于相关数据，这在资源库级代码生成等设置中非常有用。

数据集 我们在以下长语境语言建模数据集上进行了评估：PG-19（英文书籍）、arXiv（数学论文）、GitHub（代码）和 Isabelle（形式化证明）。PG-19 [Rae 等人, 2019] 是一个大型数据集，包含 1919 年以前出版的英文书籍，来源于古腾堡计划档案库。该数据集是评估长语境语言模型的公认基准[Sun 等人, 2021]。arXiv 数据集包含标注为“数学”的论文的 LATEX 源，这些论文是通过 arXiv 批量数据访问（Bulk Data Access）下载文章获得的。该数据集中每篇论文的标记数与 PG19 中一本书的标记数相当。有关其余数据集的详细信息，请参阅附录 H。

5.2 FOT 微调和上下文长度外推法

FOT 是对标准变换器架构的最小修改；因此，我们可以对现有模型进行微调，通过记忆注意力层赋予它们更长的上下文长度，这在第 4.1 节中我们已经演示过了。在本节中，我们将通过研究各种数据集的复杂度改进来深化这一分析（在一个较小的模型上）。

作为基础模型，我们使用标准目标对上下文为 1K 的 100k 步进行预训练，并使用 FOT 目标进行微调（即交叉批处理）。用于微调和预训练的数据是 Raffel 等人[2019a]的 C4 数据集（我们省略了短于 2K 字节的文档）。微调阶段需要 10k 步。我们使用的交叉批次维度为 $d=128$ ，本地上下文为 1K 标记（训练期间上下文为 2K）。我们在 4 个需要长上下文的语言建模数据集上对模型进行了零点评估：arXiv、PG-19、GitHub 和 Isabelle，详见第 5.1 节和附录 E。

从表 3 中我们可以看出，尽管 FOT 只对 2K 的可变上下文总长度进行了微调，但它在 64K 字节以内都能获得稳定的可解性收益。我们将该模型的易迷惑性与以下基线进行了比较：Memorizing Transformer (MT) [Wu 等人, 2022] 微调后的本地上下文为 1K，内存大小为 16K；Transformer-XL [Dai 等人, 2019] 微调后的本地上下文和窗口长度均为 1K。为了确保比较的公平性，所有三个模型都是从同一个基本检查点进行微调的。在使用 2K 上下文进行评估时，我们的方法取得了与 Transformer-XL 基线相当的结果，与 MT 和 FOT 不同，Transformer-XL 基线可以在所有层中访问之前的上下文。与 MT 基线相比，我们的方法在使用 64K 上下文长度进行评估时取得了更好的扩展性，并且迷惑度值也显著提高。与 MT 不同的是，我们的方法不需要在长序列上进行训练，这一点可以从 FOT 在零镜头设置下评估时较低的易错性中反映出来。更多详情，请参见附录 G。

我们还利用合成字典查询任务证实了上下文推断能力。在这项任务中，首先向模型提供 $k_i: v_i$ 映射，然后询问特定键关联的值是什么。我们使用长度为 512 的文档训练了 3700 万个参数模型。图 10 显示

表 3：微调标准变压器模型后不同上下文长度的复杂度。在 C4 上使用 FOT 目标（即交叉批处理）对模型进行了微调，并通过改变上下文大小进行了零射频评估。Transformer-XL [Dai 等人，2019 年] 和 Memorizing Transformer [Wu 等人，2022 年] 在相同设置下微调后作为基线。

方法	上下文长度	GitHub	伊莎贝尔	arXiv	PG-19
FOT	2K	6.72	5.63	8.17	23.74
	(*) (x)	5.88	4.93	7.44	23.25
	16K	5.43	4.51	6.94	22.85
	64K	5.32	4.44	6.81	22.65
变压器-XL	2K	6.85	5.76	8.21	23.57
记忆变压器	2K	8.10	7.34	9.39	24.03
	(*) (x)	7.55	6.93	8.95	23.62
	16K	7.27	6.66	8.66	23.32
	64K	7.26	6.64	8.60	23.24

经过 5k 步训练后，FOT 可以有效利用由 1600 万个 token 组成的内存，准确率超过 92%。详情见附录 F。

5.3 处理语言建模任务中的干扰

在本节中，我们将衡量在多文档环境中处理分心对语言建模的帮助。我们选取了 PG-19 数据集 [Rae 等人，2019]，并测量了在改变多文档内存（本例中由书籍组成）大小时，下一个标记预测（语言建模任务）的复杂度。直观地说，与当前书籍相对应的记忆标记可能是有益的（这一点在 [Wu et al., 2022]），中也得到了证实而来自其他书籍的记忆标记不太可能有用，因此会分散注意力。

我们观察到（见图 8），交叉批次维度 d 值越高，困惑度越高。这与第 3.3 节中的观察结果一致，表明通过减少干扰问题，我们可以在语言建模中获益。

此外，所有版本的 FOT 都能利用内存，并获得比标准变形器（无内存）更好的困惑度。不出所料，困惑度会随着内存大小的增加而增加，但我们要强调的是，这种情况发生得很优雅。在 FOT 的标准变体（粗体线）中，当扩展到 $> 500k$ 字节时，可解性只增加了 0.18。重要的是，FOT 的困惑度接近使用单文档记忆的 Memorizing Transformer 的困惑度，我们将其视为软下限，因为它不会受到无关书籍的干扰。

5.4 单文档中的上下文长度外推法

FOT 背后的初衷是通过处理干扰来提高多文档设置的性能。有趣的是，即使在单文档环境中进行评估，我们的方法也有助于推断出更长的上下文。

为了研究这一点，我们对 FOT 进行了微调（如第 5.2 节所述），并在 PG-19 数据集上以零点方式评估了不同上下文长度下所生成模型的易错性。为了深化分析，我们引入了一个附加参数 w （交叉批量训练过程中使用的先前上下文数量）。我们提供了 $w=1$ （FOT 的标准设置，相当于可微分上下文总数为 $2 \cdot 1024$ ）和 $w=2$ （相当于可微分上下文总数为 $3 \cdot 1024$ ）的结果。

见图 9，，我们观察到当上下文增长时，甚至远远超过训练上下文的长度时，情况会有所改善，这再次证实了 FOT 有助于外推到更长上下文的假设。此外， $d=2$ 明显优于 $d=1$ 。将 $d=1$ 和 $w=2$ 与 $d=2$ 和 $w=1$ 相比，我们发现前者略胜一筹。这很自然，因为前者的训练语境更长。

5.5 消融和设计选择

在附录 C 中，我们对我们的设计选择进行了分析。我们特别指出了可微分性和包含负数的重要性。我们还讨论了 Memorizing Transformer 的关系。我们注意到，由于资源有限，我们在内存层的选择上遵循了 Memorizing Transformer。

6 局限性和未来工作

我们的研究为今后的工作开辟了一些途径。我们将列出它们以及面临的挑战和局限性。

扩展上下文 这是迄今为止最重要的未来研究方向。我们面临的挑战来自于纯粹的工程设计，存储超过 1600 万个（键、值）对将需要一个分布式多节点系统。在我们的实验中，我们使用了精确的 kNN 搜索，但这种方法无法扩展到大内存。使用近似 kNN 搜索需要大量的工程设计工作，还需要仔细评估近似对模型性能的影响。

扩大交叉批处理的规模 我们发现，增加 d 是有益的。在实验中，我们使用了 $d=64$ 或 $d=128$ ，这是单个 TPUv3/TPUv2 机器内存所能容纳的最大值，参见附录一。在今后的工作中，我们希望进一步增加 d ，并在内存更大的设备上进行测试，或利用多节点训练。我们还注意到，交叉批处理会增加训练成本，但仅限于部分层。

探索对比学习 FOT 训练受相当基本的对比学习（CL）技术启发。我们的研究表明，这改进了关键结构，从而减轻了干扰问题。我们希望其他的对比学习方法也能带来益处，例如在训练过程中利用更大的内存进行硬负挖掘（见 [Lindgren 等人，2021]）。我们将这一问题留待未来工作中解决。

与其他方法相结合 开发长语境方法是一个活跃的研究领域，参见第 2 节。我们认为，其中一些方法可以与 FOT 结合使用，从而实现互利互动。

清单 1：跨批处理的可能实现。为简化代码，我们假定每份文档占用批次中两个连续的元素。更详细的版本见附录 L。

```
# 来自其他上下文的键将被编码为本地上下文开头的键
pkey_fst= pos_encode_as_first ( xk= key )

# 本地上下文键以标准方式编码 pquery , pkey= pos_encode ( xq=
query , xk= key )

# 对于批次中的 每个元素，我们计算 # 将用于交叉 - 批量 cross_batch_rel_ids=
jnp . arange ( 0 , - num_attentions , -1)
        . reshape ( 1 , -1)
batch_ids= jnp . arange ( 0 , batch_size ). reshape ( -1 , 1)
cross_batch_selector= cross_batch_rel_ids batch_ids+

# 这里我们需要其他上下文
cross_batch_keys= pkey_fst [ cross_batch_selector [: , 1:]] .

# 这里我们将本地上下文与其他上下文连接起来 attention_keys =
        jnp . concatenate ( [ pkey [: , None ] , cross_batch_keys ] , axis =1)

cb_attn_weights          = jnp .
        einsum ( " bqhd , bckhd -> bqck " \
        pquery , attention_keys , precision= precision )
```

致谢和资金披露

我们衷心感谢 TPU 研究云计划，它为我们的研究提供了重要的计算资源。项目的部分内容是利用波兰 skie Centrum Superkomputerowo - Sieciowe 的资源实现的。我们还要感谢 Markus Rabe 对初稿的审阅，以及 Christian Szegedy、Charles Staats 和 DeLesley Hutchins 的有益讨论。我们还要感谢耿新阳和刘浩发布了 OpenLLaMA 检查点和 EasyLM 库[耿，2023]，从而可以训练这些模型，这大大加快了我们的研究。Piotr Miłoś得到了波兰国家科学中心 2019/35/O/ST6/03464 号基金的支持。Henryk Michalewski 由波兰国家科学中心 UMO-2018/29/B/ST6/02959 号基金资助。

参考文献

- Joshua Ainslie、陶磊、Michiel de Jong、Santiago Ontañón、Siddhartha Brahma、Yury Zemlyanskiy、David C. Uthus、Mandy Guo、James Lee-Thorp、Yi Tay、Yun-Hsuan Sung 和 Sumit Sanghai。Colt5：带条件计算的更快长程变换器。DOI: 10.48550/arXiv.2303.09752。URL <https://doi.org/10.48550/arXiv.2303.09752>.
- Iz Beltagy、Matthew E Peters 和 Arman Cohan。Longformer：长文档转换器。*arXiv preprint arXiv:2004.05150*, 2020.
- Jonathan Berant、Andrew Chou、Roy Frostig 和 Percy Liang。从问答对在 Freebase 上进行语义解析。《2013 年自然语言处理实证方法会议论文集》，第 1533-1544 页，美国华盛顿州西雅图，2013 年 10 月。计算语言学协会。URL <https://www.aclweb.org/anthology/D13-1160>.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy、Jacob Menick、Roman Ring、Tom Hennigan、Saffron Huang、Loren Maggiore、Chris Jones、Albin Cassirer、Andy Brock、Michela Paganini、Geoffrey Irving、Oriol Vinyals、Simon Osindero、Karen Simonyan、Jack W. Rae, Erich Elsen, and Laurent Sifre。通过从数万亿词条中检索来证明语言模型。In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato, editors, *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pages 2206-2240.PMLR, 2022。URL <https://proceedings.mlr.press/v162/borgeaud22a.html>.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei。语言模型是少量学习者 *CoRR*, abs/2005.14165, 2020。URL <https://arxiv.org/abs/2005.14165>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser、默罕默德-巴伐利亚、克莱门斯-温特、菲利普-蒂莱、费利佩-彼得罗斯基 苏奇、戴夫-卡明斯、马蒂亚斯-普拉珀特、福蒂奥斯-尚齐斯、伊丽莎白-巴恩斯、阿里尔-赫伯特-沃斯、威廉-赫伯根-古斯、亚历克斯-尼科尔、亚历克斯-帕诺、尼古拉斯-特扎克、唐杰、伊戈尔-巴布什金、苏奇尔-巴拉吉、尚塔努-詹恩、威廉-桑德斯、克里斯托弗-赫塞、安德鲁-N.Carr、Jan Leike、Joshua Achiam、Vedant Misra、Evan Morikawa、Alec Radford、Matthew Knight、Miles Brundage、Mira Murati、Katie Mayer、Peter Welinder、Bob McGrew、Dario Amodei、Sam McCandlish、Ilya Sutskever 和 Wojciech Zaremba。评估基于代码训练的大型语言模型。《CoRR》, abs/2107.03374, 2021。URL <https://arxiv.org/abs/2107.03374>.
- 陈寿元、黄世文、陈良健、田远东。通过位置插值扩展大型语言模型的上下文窗口，2023年。

- Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 视觉表征对比学习的简单框架。 *机器学习国际会议*, 第 1597-1607 页。PMLR, 2020.
- 戴子航、杨志林、杨一鸣、Jaime Carbonell、Quoc V Le 和 Ruslan Salakhutdinov。Transformer-xl: Attentive language models beyond a fixed-length context. *ArXiv preprint arXiv:1901.02860*, 2019.
- Tri Dao、Daniel Y. Fu、Stefano Ermon、Atri Rudra 和 Christopher Ré。Flashattention: 快速且内存效率高的精确注意力与生物感知。在 2022 年 *NeurIPS* 大会上。URL http://papers.nips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.
- Pradeep Dasigi、Kyle Lo、Iz Beltagy、Arman Cohan、Noah A. Smith 和 Matt Gardner。研究论文中的信息搜索问题和答案数据集, 2021 年。
- 斯特凡·埃尔夫温、内部荣治和卢谷健二。强化学习中神经网络函数近似的西格玛加权线性单元》, 2017年。
- 高俊、何迪、谭旭、秦涛、王立伟和刘铁岩。训练自然语言生成模型中的表征退化问题, *arXiv preprint arXiv:1907.12009*, 2019.
- Leo Gao、Jonathan Tow、Stella Biderman、Sid Black、Anthony DiPofi、Charles Foster、Laurence Golding、Jeffrey Hsu、Kyle McDonnell、Niklas Muennighoff、Jason Phang、Laria Reynolds、Eric Tang、Anish Thite、Ben Wang、Kevin Wang 和 Andy Zou。少量语言模型评估框架》, 2021 年 9 月 a。URL <https://doi.org/10.5281/zenodo.5371628>.
- 高鲁豫、张云翼、韩家炜和杰米·卡伦。在内存受限设置下扩展深度对比学习批量大小。 *arXiv preprint arXiv:2101.06983*, 2021b.
- Xinyang Geng. EasyLM: 大型语言模型的简单可扩展训练框架, 2023。URL <https://github.com/young-geng/EasyLM>.
- 耿新阳、刘浩。Openllama: llama 的开放再现, 2023 年 5 月。URL https://github.com/openlm-research/open_llama.
- Mandy Guo、Joshua Ainslie、David Uthus、Santiago Ontanon、Jianmo Ni、Yun-Hsuan Sung 和 Yinfei Yang。Long5: Efficient text-to-text transformer for long sequences. *ArXiv preprint arXiv:2112.07916*, 2021.
- 郝亚茹、孙玉涛、董莉、韩志雄、顾玉贤、魏福如。结构化提示: 将上下文学习扩展到 1,000 个示例。 *CoRR*, abs/2212.06713, 2022。DOI: 10.48550/arXiv.2212.06713。URL <https://doi.org/10.48550/arXiv.2212.06713>.
- Adi Haviv、Ori Ram、Ofir Press、Peter Izsak 和 Omer Levy。无位置编码的变换器语言模型仍能学习位置信息》, 2022 年。
- Alex Henry、Prudhvi Raj Dachapally、Shubham Shantaram Pawar 和陈宇轩。变换器的查询密钥规范化。在特雷弗·科恩、何玉兰和刘洋主编的《计算语言学协会的研究成果》 (*Findings of the Association for Computational Linguistics: EMNLP 2020*, 在线活动, 2020 年 11 月 16-20 日, *ACL 研究成果 EMNLP 2020* 卷, 第 4246-4253 页。doi: 10.18653/v1/2020.findings-emnlp.379。URL <https://doi.org/10.18653/v1/2020.findings-emnlp.379>.
- Eduard Hovy、Laurie Gerber、Ulf Hermjakob、Chin-Yew Lin 和 Deepak Ravichandran。基于语义的答案精确定位。 *第一届人类语言技术研究国际会议论文集*, 2001 年。URL <https://www.aclweb.org/anthology/H01-1069>。
- Nihal Jain、Dejiao Zhang、Wasi Uddin Ahmad、Zijian Wang、Feng Nan、Xiaopeng Li、Ming Tan、Ramesh Nallapati、Baishakhi Ray、Parminder Bhatia、Xiaofei Ma、and Bing Xiang。ContracLM: 因果语言模型的对比学习, 2023年。

Albert Qiaochu Jiang, Wenda Li, Szymon Tworkowski, Konrad Czechowski, Tomasz Odrzygóź'dź, Piotr Miłoś, Yuhuai Wu, and Mateja Jamnik. 雷神：挥舞锤子整合语言模型和自动定理证明器。见 Alice H. Oh、Aleksh Agarwal、Danielle Belgrave 和 Kyunghyun Cho 编辑，《神经信息处理系统进展》，2022 年。URL <https://openreview.net/forum?id=fUeOyt-2EOp>.

Jeff Johnson、Matthijs Douze 和 Hervé Jégou. 使用GPUS的亿级相似性搜索》，2017年。

Jean Kaddour、Oscar Key、Piotr Nawrot、Pasquale Minervini 和 Matt J. Kusner. 没有训练就没有收获：重新审视基于转换器的语言模型的高效训练算法》，2023 年。

kaiokeudev. Things in learning while training superhot. https://kaiokeudev.github.io/til#_extending-context-to-8k, 2023.

Urvashi Khandelwal、Omer Levy、Dan Jurafsky、Luke Zettlemoyer 和 Mike Lewis. 通过记忆实现泛化： *ArXiv preprint arXiv:1911.00172*, 2019.

Denis Kocetkov、Raymond Li、Loubna Ben Allal、Jia Li、牟成浩、Carlos Muñoz Ferrandis、Yacine Jernite、Margaret Mitchell、Sean Hughes、Thomas Wolf、Dzmitry Bahdanau、Leandro von Werra 和 Harm de Vries. 堆栈3 tb 的许可源代码。预印本，2022 年。

工藤拓和约翰·理查森. SentencePiece：用于神经文本处理的简单且独立于语言的子词标记器和去标记器。 *自然语言处理实证方法 2018 年会议论文集：系统演示*，第 66-71 页，比利时布鲁塞尔，2018 年 11 月。DOI: 10.18653/v1/D18-2012. URL <https://aclanthology.org/D18-2012>.

Aitor Lewkowycz, Anders Johan Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Venkatesh Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. 用语言模型解决定量推理问题。见 Alice H. Oh、Aleksh Agarwal、Danielle Belgrave 和 Kyunghyun Cho 编辑，《神经信息处理系统进展》，2022 年。URL <https://openreview.net/forum?id=IFXTZERXdm7>.

Xin Li and Dan Roth. 学习问题分类器。在 *COLING 2002：第19届计算语言学国际会议*，2002 年。URL <https://www.aclweb.org/anthology/C02-1150>.

Yujia Li, David H. Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland, Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 用alphacode生成竞赛级代码。DOI: 10.48550/arXiv.2203.07814.

Erik Lindgren、Sashank Reddi、Ruiqi Guo 和 Sanjiv Kumar. 使用负缓存高效训练检索模型。见 M. Ranzato、A. Beygelzimer、Y. Dauphin、P.S. Liang 和 J. Wortman Vaughan 编辑的《神经信息处理系统进展》第 34 卷第 4134-4146 页。Curran Associates, Inc., 2021。URL https://proceedings.neurips.cc/paper_files/paper/2021/file/2175f8c5cd9604f6b1e576b252d4c86e-Paper.pdf.

Dacheng Li Rulin Shao Anze Xie Ying Sheng Lianmin Zheng Joseph E. Gonzalez Ion Stoica Xuezhe Ma and Hao Zhang. 开源 llms 真正能承诺的上下文长度有多长？，2023 年 6 月。URL <https://lmsys.org/blog/2023-06-29-longchat>.

Maciej Mikula、Szymon Antoniak、Szymon Tworkowski、Albert Qiaochu Jiang、Jin Peng Zhou、Christian Szegedy、Łukasz Kucinski、Piotr Miłoś 和 Yuhuai Wu. Magnushammer：基于变压器的前提选择方法，2023 年。

Amirkeivan Mohtashami 和 Martin Jaggi. 地标关注：变压器的随机访问无限上下文长度。 *CoRR*, abs/2305.16300, 2023. DOI: 10.48550/arXiv.2305.16300. URL <https://doi.org/10.48550/arXiv.2305.16300>.

MosaicML。介绍 mpt-30b：提高开源基础模型的标准，2023 年。网址 www.mosaicml.com/blog/mpt-30b。访问时间：2023-06-22。

Piotr Nawrot, Szymon Tworowski, Michal Tyrolski, Lukasz Kaiser, Yuhuai Wu, Christian Szegedy, and Henryk Michalewski. 分层变换器是更高效的语言模型。 *CoRR*, abs/2110.13711, 2021。URL <https://arxiv.org/abs/2110.13711>。

Piotr Nawrot, Jan Chorowski, Adrian Kupczuk 和 Edoardo M. Ponti。具有动态令牌池的高效变压器，2023 年。

Erik Nijkamp, Hiroaki Hayashi, Tian Xie, Congying Xia, Bo Pang, Rui Meng, Wojciech Kryscinski, Lifu Tu, Meghana, Chen Xing, Jesse Vig, Lidiya Murakhovska, Chien-Sheng Wu, Silvio Savarese, Yingbo Zhou, Shafiq Rayhan Joty 和 Caiming Xiong。用 xgen 进行长序列建模：在 8k 输入序列长度上训练的 7b llm。Salesforce AI 研究博客，2023 年。URL <https://blog.salesforceairesearch.com/xgen-7b/>。

Stanislas Polu 和 Ilya Sutskever。用于自动定理证明的生成语言建模。 *CoRR*, abs/2009.03393, 2020。URL <https://arxiv.org/abs/2009.03393>。

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 语言模型是无监督的多任务学习者。 *OpenAI blog*, 1(8):9, 2019。

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. *国际机器学习大会*, 第 8748-8763 页。PMLR, 2021 年。

Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, Chloe Hillier, and Timothy P Lillicrap。用于长序列建模的压缩变换器。 *ArXiv 预印本*, 2019 年。URL <https://arxiv.org/abs/1911.05507>。

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li 和 Peter J. Liu。用统一的文本到文本转换器探索迁移学习的极限。 *ArXiv e-prints*, 2019a。

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li 和 Peter J. Liu。用统一的文本到文本转换器探索迁移学习的极限。 *CoRR*, 2019年10月b。

Nir Ratner, Yoav Levine, Yonatan Belinkov, Ori Ram, Inbal Magar, Omri Abend, Ehud Karpas, Amnon Shashua, Kevin Leyton-Brown 和 Yoav Shoham。大型语言模型的并行上下文窗口》，2023 年。

乌里-沙哈姆、埃拉德-西格尔、莫尔-伊夫吉、阿维亚-埃弗拉特、奥里-约兰、阿迪-哈维夫、安基特-古普塔、熊文汉、莫尔-格瓦、乔纳森-贝兰特和奥马尔-列维。卷轴：长语言序列的标准化比较，2022 年。

Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu。Roformer：具有旋转位置嵌入的增强型变压器。 *CoRR*, abs/2104.09864, 2021。url <https://arxiv.org/abs/2104.09864>。

孙思萌、Kalpesh Krishna、Andrew Mattarella-Micke、Mohit Iyyer。长程语言模型真的会使用长程语境吗？

TogetherComputer.Redpajama：重现 骆 马 训 练 数 据 集 的 开 源 配 方，2023 年。URL <https://github.com/togethercomputer/RedPajama-Data>。

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *ArXiv preprint arXiv:2302.13971*, 2023。

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin。关注就是一切。 *CoRR*, abs/1706.03762, 2017。URL <http://arxiv.org/abs/1706.03762>。

吴雨怀、马库斯-诺曼-拉贝、德莱斯利-哈钦斯和克里斯蒂安-塞格迪。记忆反式表征。第十届学习表征国际会议, *ICLR 2022, 虚拟活动, 2022年4月25-29日*。OpenReview.net, 2022.URL <https://openreview.net/forum?id=TrjbxzRcnf->。

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: 用于较长序列的变压器。《*神经信息处理系统进展*》, 33:17283-17297, 2020年。

Biao Zhang 和 Rico Sennrich.均方根层归一化, 2019。

Zexuan Zhong, Tao Lei, and Danqi Chen.利用记忆增强训练语言模型载于 Yoav Goldberg、Zornitsa Kozareva 和 Yue Zhang 编辑的《2022年自然语言处理经验方法会议论文集》(EMNLP 2022), 阿拉伯联合酋长国阿布扎比, 2022年12月7-11日, 第5657-5673页。计算语言学协会, 2022年。网址 <https://aclanthology.org/2022.emnlp-main.382>。

更广泛的影响

最近，语言模型的快速发展带来了许多新功能。与此同时，这些新功能也引发了人们对其社会影响的担忧，并在社会上引起了热烈的讨论。我们的工作开发了一种通用技术，原则上可以应用于几乎任何语言模型，从而通过扩展其功能来加剧威胁。不过，我们注意到，FOT 并没有产生任何新的威胁。因此，我们参考了关于语言模型更广泛影响的现有知识体系，如 Borgeaud 等人的研究[2022]。

A 龙拉马

A.1 架构

OpenLLaMA [Geng and Liu, 2023] 是 LLaMA [Touvron et al., 2023] 的。开源复制品它使用旋转位置嵌入的纯解码器架构，并做了一些改动，包括使用 RMSNorm [Zhang and Sennrich, 2019] 进行预规范化，以及 SiLU 激活 [Elfwing 等人, 2017]。使用了具有 32k 词汇量的 SentencePiece [Kudo and Richardson, 2018] 标记化器。

A.2 用 FOT 扩展上下文长度

位置编码 为了实现与原始 LLaMA 的向后兼容，我们在本地上下文中保留了位置编码。本地上下文之外的标记被分配到与本地上下文中第一个标记相同的位置。

对较长的上下文进行密集关注 为了简化实现过程，减少对外部软件的依赖，我们不再使用 kNN 查找，而是对整个内存进行关注。我们发现，这两种内存关注方法之间的性能差异很小。

交叉批处理细节 对于 3B LONGLLAMA 模型，我们将 $L = \{6, 12, 18\}$ 设置为内存层。我们通过将批次条目分为大小相等的四个片段，来改变批次元素的额外上下文 $d \in \{0, 2, 3\}$ 的数量。第一段的元素只能看到本地上下文 ($d = 0$)。第二段的元素会看到两个附加上下文 ($d = 2$)，一个来自同一文档（正面），另一个来自不同文档（负面）。第三部分的元素会看到三个额外的上下文，其中两个是正面的，一个是负面的。最后一个部分由元素组成，它们会看到来自同一文档的三个额外语境。我们将这种设置简称为

$$\frac{(1)(0)}{4}, 0), \frac{(1)(1)}{4}, 1), \frac{(1)(2)}{4}, 1), \frac{(1)(3)}{4}, 0).$$

对于 7B LONGLLAMA 模型，我们将 $L = \{8, 16, 24\}$ 设置为存储层。在此，我们将批次条目分为四段，并使用以下设置： $\frac{(1)(0)}{4}, 0), \frac{(1)(1)}{4}, 2), \frac{(1)(2)}{4}, 5), \frac{(1)(3)}{4}, 4).$

超参数 我们沿用了 OpenLLaMA 在大部分超参数方面的选择，包括使用相同的优化器。在微调过程中，我们使用的批量大小为 256K 标记，恒定学习率为 $2e-5$ ，低于 OpenLLaMA 训练结束时的学习率（1T 标记后为 $3e-5$ ），权重衰减为 0.01。

A.3 LLaMA 微调数据集

我们使用基于 RedPajama [TogetherComputer, 2023] 和 The Stack [Kocetkov 等人, 2022] 的混合数据集，每个子集的比例如下：

除 python 之外的所有子集都直接取自 RedPajama。对于 python 子集，我们从 The Stack 收集 Python 源代码，并采用与 H 节中 GitHub 数据集类似的方法，将同一子目录下的文件以随机顺序串联起来，从而获得长文档用于训练。此外，我们还过滤掉了原始 RedPajama 中某些子集的短文档，即短于最小文档长度一栏所示的文档。

如果某个文档太短，无法跨越多个上下文进行交叉批处理，那么我们会将其与数据集中的下一个文档连接起来。

表 4: 用于 LONGLLAMA 微调混合物的 RedPajama 子集比例。对于 python 子集, 我们使用了来自 The Stack 的数据 (详见正文)。如果有指定, 我们只对长度至少为最小文档长度的文档进行训练 (否则, 我们对该子集的所有文档进行训练)。水平线分隔了长上下文子集和短上下文子集。

子集	取样比例 (%)	最小文档长度
arxiv	25	-
python	25	4096
书籍	10	-
共同爬行	29	-
c4	5	1024
github	2	2048
stackexchange	2	1024
维基百科	2	1024

A.4 语言模型评估工具箱

为了确保 LONGLLAMA_s 的性能在短语境场景中没有下降, 我们在语言模型评估工具包基准[Gao 等人, 2021a]。上对我们的模型进行了评估表 5 比较了我们与 OpenLLaMA [Geng and Liu, 2023] 的。结果与 OpenLLaMA 的作者类似, 我们省略了 CB 和 WSC 任务。

表 5: 在语言模型评估工具包 (Language Model Evaluation Harness) 上对不同任务/指标的模型进行比较。LONGLLAMA 模型在没有语境扩展的情况下进行了评估 (即作为标准 OpenLLaMA 模型)。结果表明, LONGLLAMA 在短语境场景中保持了良好的性能。

任务/指标	OpenLLaMA 3B	LongLLaMA 3B	OpenLLaMA 7B	LongLLaMA 7B
anli_r1/acc	0.33	0.32	0.33	0.35
anli_r2/acc	0.32	0.33	0.36	0.37
anli_r3/acc	0.35	0.35	0.38	0.36
arc_challenge/acc	0.34	0.34	0.37	0.37
arc_challenge/acc_norm	0.37	0.37	0.38	0.38
arc_easy/acc	0.69	0.68	0.72	0.70
arc_easy/acc_norm	0.65	0.63	0.68	0.66
boolq/acc	0.68	0.68	0.71	0.71
hellaswag/acc	0.49	0.48	0.53	0.52
地狱袋/公制	0.67	0.65	0.72	0.71
openbookqa/acc	0.27	0.28	0.30	0.30
openbookqa/acc_norm	0.40	0.38	0.40	0.41
piqa/acc	0.75	0.73	0.76	0.75
piqa/acc_norm	0.76	0.75	0.77	0.76
record/em	0.88	0.87	0.89	0.89
记录/f1	0.89	0.87	0.90	0.90
RTE/acc	0.58	0.60	0.60	0.59
truthfulqa_mc/mc1	0.22	0.24	0.23	0.24
truthfulqa_mc/mc2	0.35	0.38	0.35	0.35
WIC/ACC	0.48	0.50	0.51	0.50
winogrande/acc	0.62	0.60	0.67	0.67
平均得分	0.53	0.53	0.55	0.55

A.5 研究论文的问题解答

我们在 SCROLLS [Shaham 等人, [等2022] 人, 2021。的 Qasper Dasigi] 验证集上评估了模型的上下文利用率详情

见表 6。

表 6: 在 Qasper 验证子集[Dasigi 等人, 2021]。上使用不同上下文长度时的归零性能我们使用了语言模型评估工具箱 ([Gao et al.Language Model Evaluation Harness) 在 Harness 的实现中, "是/否 "问题与开放式问题是分开评估的。请注意, LONGLLAMA 3B 受益于扩展的语境。

语境长度	OpenLLaMA 3B	LONGLLAMA 3B	LLaMA 7B	长聊 7B
2K	18.7	18.7	18.7	19.4
4K	-	20.7	-	21.2
6K	-	23.2	-	25.0
8K	-	26.6	-	28.8

B 建筑

本节介绍本文所介绍的非基于 LLaMA 的模型的结构和交叉批处理细节。主要区别在于, 对于基于 LLaMA 的模型 (LONGLLAMA), A.2), 我们保留了位置编码 (稍作修改, 详见 没有引入注意力温度参数, 并用完全密集注意力取代了 kNN。

B.1 转换器模型

对于非基于 LLAMA 的模型, 我们使用 [Vaswani 等人, 2017] 中介绍的变换器架构, 并做了一些标准的修改。首先, 我们只使用解码器, 而不使用编码器部分。其次, 我们在注意力模块和前馈模块输入之前进行层归一化。此外, 我们还使用了旋转位置嵌入技术 (Rotary [Su 等Position Embedding) 人, 2021 年], 人, 2020 年], 对键和查询进行了归一化处理[Henry 等并为每个注意力头引入了一个可学习的温度参数。

每个模型大小的超参数见附录 E。在 PG-19 上训练模型时, 我们使用了 32k 词汇量的标准 T5 标记化器[Raffel 等人, 2019b]。第 5.2 节中的大型模型则是用定制的 SentencePiece 标记器[Kudo and Richardson, 2018]训练的, 词汇量为 64k。

B.2 记忆注意层

记忆注意层 ℓ 是转换层之一, 它可以访问附加语境 M 。内存存储 (键、值) 对。对于 ℓ 中的每个查询 q , 我们都会从 M 中检索出 k 个最匹配的条目, 并用它们来计算注意力值。更准确地说, 我们使用 kNN 算法选取 $M_{(top)} := \{(key_{(1)}, value_{(1)}), \dots, (key_k, value_k)\} \subset M$, 这样 $\{(q, key_{(i)})\}_{(i=1) \dots k}$ 就是 M 中的前 k 个内积。这些内积与 q 之前的本地上下文部分合并, 记为 $C_{<q}$, 并使用标准变换器公式计算注意力值:

$$v := \frac{\sum_{(key, v) \in M_{top} \cup C_{<q}} s(key) \cdot v}{\sum_{(key, v) \in M_{top} \cup C_{<q}} s(key)}, \quad (1)$$

其中, $s(key)$ 是 key 的软最大得分。软最大值的计算方法如下:

$$softmax(q, \frac{key}{\tau}) = \frac{\exp(\frac{q \cdot key}{\tau})}{\sum_{key \in M_{top} \cup C_{<q}} \exp(\frac{q \cdot key}{\tau})},$$

其中, τ 是一个温度参数。在这种方法中, 我们不区分本地上下文和内存。

另一种整合 M_{top} 的方法是通过门控。在这种方法中, 我们分别计算 M_{top} 和本地上下文 v_C 的注意力值 $v_{(M)}$ (使用标准 Transformer 公式)。然后, 我们使用门控机制将它们结合起来:

$$v := v_M \cdot g + v_C \cdot (1 - g), \quad g = \sigma(b_g),$$

其中, σ 为 sigmoid 函数, b_g 为可训练偏置。门控方法在[Wu 等人, 2022]中提出, 见公式[Wu 等人, 2022, (2)]。

我们发现我们的方法，即使用 (1) 也同样有效，见图 4。同时，(1) 更为简单，不需要额外参数。因此，我们在实验中使用了它。

对于 kNN 查找，我们使用 FAISS [Johnson 等人, 2017]。中实现的精确 kNN 搜索记忆注意层不使用位置编码。内存是由事先经过 ℓ 处理的 (键, 值) 对增量填充的。在单文档设置中，内存会在每个文档结束后被清除。

对于 LONGLLAMAs，我们不使用 τ 参数，因为它们的架构不会对键和查询进行规范化处理。对于 LONGLLAMAs，我们还用密集注意力替换了 kNN 搜索，并保留了位置编码（见附录 A.2）。

B.3 交叉批量训练程序

在 FOT 中，我们选择注意力层的一个子集 L ，以便以后使用 (键, 值) 对的记忆进行扩充。在训练过程中，我们会让该层接触到来自当前本地上下文 C_{curr} 和上一个本地上下文以及来自其他文档的 $d-1$ 上下文 C_{prev} 的 (键, 值) 对的混合物；图示另见图 2。为此，我们修改了输入管道，使每个批次索引对应不同的文档（每个文档所占用的批次索引从加载文档到处理完毕都是固定的）。

更具体地说，我们在批处理中嵌入了每个文档的上一个和当前本地上下文。然后，我们使用 C_{prev} 作为给定文档的上一个本地上下文和来自其他文档的 $d-1$ 个上下文的来源。对于批次中的每个元素，这 d 个额外上下文的选择是固定的。我们禁用了 ℓ 中的位置编码，因为我们设想它可以处理全局信息。

更准确地说，对于批次中的每份文档 δ 和来自层 ℓ 的查询 q ，我们都会创建一个集合 p^δ ，该集合由文档 δ 上一个本地上下文中的 (键, 值) 对以及从 C_{prev} 中收集的 $d-1$ 上下文中的对 (键, 值) 组成。 q 的关注值由以下公式给出

$$v := \sum_{(key, v) \in p^\delta \cup C_{\text{prev}}^{(\delta, < q)}} s(key) \cdot v, \quad (2)$$

其中， $C_{\text{prev}}^{(\delta, < q)}$ 由本地上下文中 q 前面的 (key-value) 对组成， $s(key)$ 是 key 的 softmax 分数。我们使用具有可学习温度 τ 的 softmax：

$$softmax(q, key) = \frac{\exp(\frac{q \cdot key}{\tau})}{\sum_{key \in p^\delta \cup C_{\text{prev}}^{(\delta, < q)}} \exp(\frac{q \cdot key}{\tau})}.$$

请注意，(1) 和 (2) 之间的唯一区别是附加 (键, 值) 对的来源： p^δ 。这尤其意味着所有与前文相关的运算都是可微分的。

不同文档的数量等于 $b(s)$ 批量大小，即每个文档在批量中都有一个单独的索引）。 C_{prev} 中批次索引在 $\{i, (i+1) \bmod b(s), \dots, (i+d-1) \bmod b(s)\}$ 中的所有标记纳入 p^δ ， $(i+d-1) \bmod b(s)$ 。

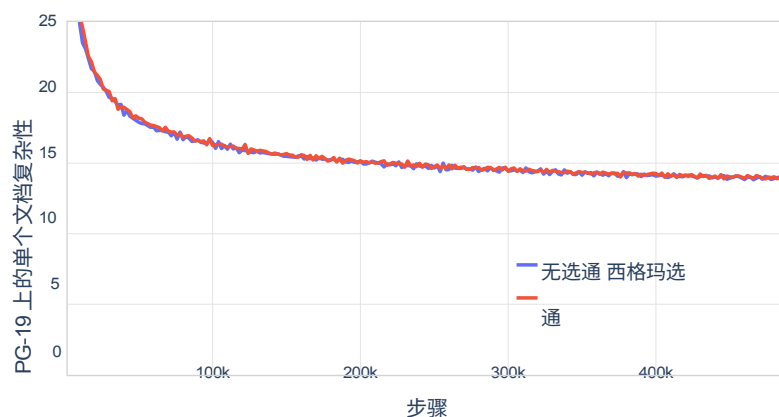


图 4: PG-19 训练过程中的复杂度（测试集）。我们训练了两个 Memorizing Transformer 模型[Wu 等人, 2022]，一个有原始门控，另一个没有（即如（1））。所述，记忆注意力与局部注意力共享我们使用 16k 内存的单文档设置。

B.4 定性分析

表 7 提供了对 FOT 的简要定性分析。它表明，该模型可以处理干扰，并从 PG-19 任务数据集中的多文档存储器中检索字符名称的部分内容，以及从大型字典中检索适当的定义（字典查询任务）。

B.5 记忆变换器

聚焦变换器与记忆变换器有许多相似之处[Wu 等人, 2022]。在本节中，我们将总结这两个模型之间的差异。

训练 这两种方法的关键区别在于训练过程。我们的方法使用交叉批处理（参见 B.3），节简而言之，这是标准的转换器训练目标，但我们会额外关注上一个上下文窗口中的标记，包括来自同一文档和不同文档的标记，详见附录 B.3。记忆转换器对从同一文档中检索到的标记进行训练（设想用于单文档）。

这有几个重要的后果：

- FOT 在训练过程中不使用内存，而 MT 则需要。这可能会加快训练速度；此外，FOT 总是使用最新的值，而 MT 则使用内存中的值，后者可能已经过时。
- FOT 可通过所有（键、值）对进行区分，而 MT 不会通过检索到的标记进行区分。我们认为，这是联合训练结构良好的键、值和查询嵌入的关键，因此也是获得良好模型性能的关键。
- FOT 不需要训练集中的长文档，而 MT 则需要，以便在记忆中捕捉长的依赖关系。这一点非常重要，因为许多流行的数据集都由短文档组成。

我们推测，将这两种方法融合在一起可能会有好处。例如，我们可以认为 MT 更善于为模型提供“硬”否定。我们在附录 C.3 中提供了一个概念验证实验，并将其留待未来工作中使用。

推理 这两种模型都使用非常相似的记忆注意力层。区别在于如何整合检索到的（键、值）对。FOT 采用与本地上下文相同的方式处理检索到的信息。MT 使用的是门控机制。详情见 B.2。节

表 7: kNN 搜索检索到的元素示例及其在 PG-19 和查字典任务中的得分。第一列显示了与特定查询相关的标记（粗体）及其上下文片段。第二列显示与 kNN 为该查询检索的键相关的标记。kNN 分数显示了检索到的关键字的关注度占特定关键字的比例。Focus 分数的计算方法是：将关键字与 32 个前关键字和 32 个后关键字放在一起，计算它们的注意力权重，然后检查检索到的关键字获得了多少注意力。在 PG-19 设置中，模型配备了大小为 4096 的内存，跨越 8 个文档的各个部分。在字典查找设置中，模型配备了 16M 大小的内存。

文本	kNN 结果	kNN 焦点得分	
PG-19			
她拿起挂在院长马甲上的金边耳坠，放在他眼前	那么，如果我们在这里与封闭的马车十点[他们一起走进图书馆 达比：[对SHE BA说]	0.69	0.99
	他瘫坐在沙发上，目光呆滞，双臂无力地垂下。达尔 贝：[现在，他的事业成了他的负担！	0.23	0.92
	爸爸爸爸！Papsey！[院长站起来，发现了他的孩子们，摘下帽子。 那如果我们十点钟还在这儿，马车还关着的话[他们一起走进书房 达贝-伊[致谢巴]	0.025 0.92	
	院长严厉地看了达尔贝伊一眼，然后咳嗽一声走进图书馆。男人们和女孩们在暗中议论纷纷。	0.71	0.99
院长严厉地看了达尔贝伊一眼，然后咳嗽一声走进图书馆。男人们和女孩们在暗中议论纷纷。	哦！[他瘫坐在沙发上，目光呆滞，双臂无力地垂下。 达贝-伊[现在他的事业成了他的负担！	0.19	0.99
	哦，莎乐美爸爸爸爸院长？院长？院长！院长	0.08	0.99
词典查询任务			
<q> 14 42 23 38 <v> 40 41 05 56	<k> 14 42 23 38 <v> 40 41 05 56	0.96	0.99
<Q> 30 55 07 23 <V> 36 17 26 63	<k> 30 55 07 23 <v> 36 17 26 63	0.88	1.0
<Q> 10 41 26 39 <V> 48 38 11 24	<k> 10 41 26 39 <v> 48 38 11 24	0.87	0.99

C 消融

在本节中，我们将重点讨论交叉批处理训练程序的两个关键特性：可微分性和包含负数。我们还讨论了在训练协议和记忆整合方面与 Memorizing Transformer 的关系。关于 FOT 和 Memorizing Transformer 之间差异的详细技术说明，请参阅附录 B.5。

C.1 可变键和值的影响

我们将 FOT 与 Memorizing Transformer 进行了比较，后者在训练过程中使用的是密钥和值的无差别存储器。在图 5，所示的多文档实验中 MT 和 FOT 均使用 512 的本地上下文进行训练。我们发现，在推理过程中扩展上下文时，FOT 的效果明显更好，这证明了可区分的键和值是有益的。

表 8：在单文档设置下，PG-19 在训练过程中使用不同局部上下文长度时的复杂度。在这些实验中，我们在训练和评估过程中使用了相同的上下文长度。

上下文长度	FOT d=1	MT
512	14.18	14.68
1024	14.17	14.46
2048	14.11	14.43

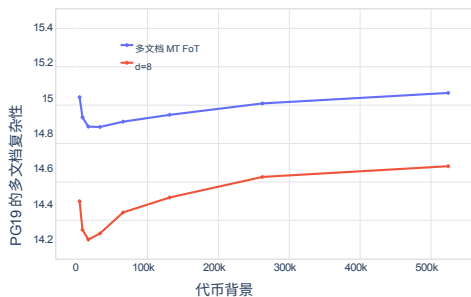


图 5：多文档集中 PG-19 的复杂度。FOT 和多文档 MT 均使用大小为 512 的本地上下文进行训练。在训练过程中，多文档 MT 使用了 8 个文档共享的大小为 4096 的内存。键和值的可区分性带来了更好的困惑度。

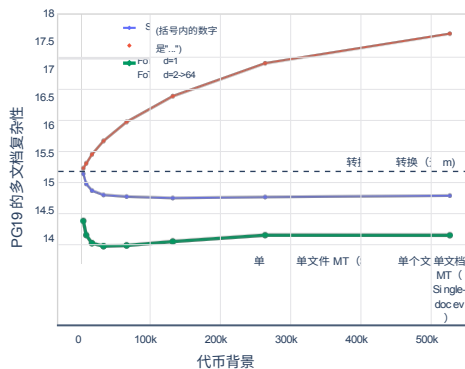


图 6：多文档设置中否定词的重要性。我们将使用 $d=1$ 训练的 FOT 与使用 $d=2$ 训练的 FOT 进行了比较，后者后来改用了 $d=64$ 。为了进行更多比较，我们还展示了使用 16K 内存训练的 MT 的性能。

我们还检查了可变密钥和值是否能提高单文档环境下的性能。为此，我们将使用 $d=1$ 的 FoT 与使用由之前本地上下文组成的内存的 MT 进行了比较。表 8 证实，可变键和值在这种情况下也有帮助。

C.2 否定词的重要性

我们重申了否定句在多文档环境中的重要性。在图 3，以前的实验中我们已经观察到，增加否定词的数量（即增加 d ）会使更多的注意力集中到相关标记上。在图 6 中，我们还发现，当上下文长度增加时，训练中缺少否定词（ $d=1$ ）会导致模型复杂度显著下降。这证明，使用否定词和可区分性对于 FOT 的良好运行都很重要。

C.3 与记忆变换器的关系

Memorizing Transformer Wu 等人研究[2022]与我们的方法密切相关。两个主要区别在于：1) 训练协议；2) 如何将记忆集成到模型中。在本节中，我们将就这些差异提供更多见解。

训练协议 在前面的章节中，我们已经讨论了跨批训练的优势，即使用对比启发目标和通过前语境进行反向传播。MT 方法的潜在优势在于，它在训练过程中能接触到整个记忆（而不仅仅是之前的语境）。我们进行了一次概念验证实验，将这两种方法结合起来，以进一步探讨这一问题。也就是说，我们使用交叉批处理对模型进行了 499k 步的训练，并使用 MT 目标对模型进行了 1k 步的微调。有趣的是，在相同的步数预算下，我们观察到与 MT 训练相比有了明显的改善，见图 7。我们相信，结合两种方法的优点，我们还可以进一步探索各种训练协议。

内存整合 FOT 采用简单的内存整合方法，通过 kNN 查找检索到的（键/值）对与本地上下文的处理方式相同。与此相反，MT 使用了一种门控机制、内存和本地值的加权平均；详见附录

B.2.我们对这两种方法进行了评估，发现这两种内存整合方法在性能上没有差别。不过，我们决定使用我们的方法，因为它不需要

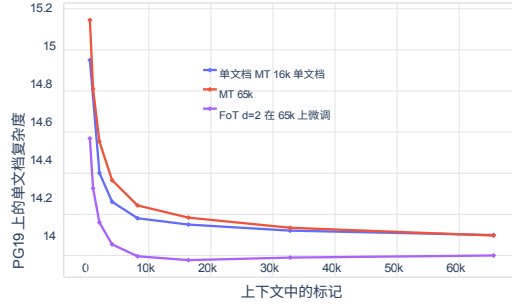


图 7: FOT 在非可微分内存上进行 1k 步微调的单文档评估。与 MT 相比, FOT 的困惑度更低, 而 MT 在整个训练过程 (500k 步) 中都可以使用该内存。

FOT 不需要做任何架构上的改动 (因此很容易对现有模型进行微调)。因此, 我们推荐使用 MT。我们推测, FOT 中不需要门控的另一个原因是, 跨批次训练可以通过上一个上下文 C_{prev} 中的 (键、值) 对进行反向传播, 而 MT 无法反向传播, 在计算键和值的梯度时需要依赖本地上下文。另一个原因可能是, C_{prev} 被嵌入到每个批次中, 从而避免了僵化 (见[Wu 等, 2022 年, 第 3.2 节])。

D 其他数据

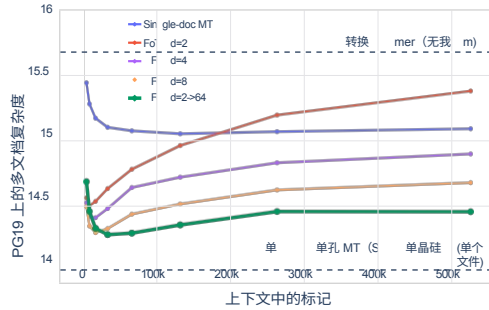


图 8: 多文档环境下的复杂度。FOT 2->64 从 $d=2$ 开始, 然后切换到 $d=64$ 。单文档 MT 的内存大小为 16K。随着内存容量的增加, 干扰项 (无关键) 的数量也会增加, 从而增加了任务的难度。在单个文档设置下评估的单个文档 MT 是一个软下限, 因为它缺乏干扰。

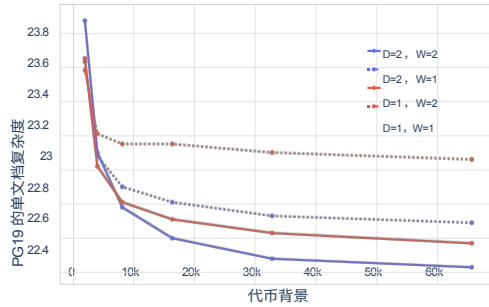


图 9: 在 C4 上经过预训练的 FOT 在 PG19 上的零帧性能。使用交叉批次维度 $d=2$ 微调的模型优于使用 $d=1$ 微调的模型。使用 2048 的双倍 ($w=2$) 训练背景是有益的。

E 超参数

表 9 显示了实验中使用的超参数。除非另有说明，我们使用的上下文长度为 512。在附录 F 第 5.3 节和第 5.5 节中我们使用了 32K 标记的总批次大小。在第 5.2 节和第 5.4 节中，总批次大小为 128K 标记。

表 9：不同模型大小的超参数。批量大小以标记数为单位。对于 37M 模型， FOT 的局部上下文长度为 256，基线为 512。

超参数	值 常见	
#层数	12	
记忆注意力层索引 (l) 优化器	8	
学习率计划 热身步骤	AdaFactor 反平方	
β_1	根	
	1000	
	0.9	
	特定模型	
#Params	37M	184M
最大学习率	0.02	0.01
最小学习率	0.01	0.0005
嵌入尺寸	512	1024
头部尺寸	64	128
# Heads	8	8
前馈尺寸	2048	4096
本地上下文长度	256/512	512
批处理大小	32K/64K	32K
#训练步数	5k	500k

在第 5.3 节和第 5.5 节所述的实验中，我们进行了以下超参数扫描：

- 学习率： $\{1e-2, 5e-3, 3e-3, 1e-3\}$ ，选择： $1e-2$ 、
- 批量大小： $\{8K, 16K, 32K\}$ ，选择：32K。

对于查字典任务（附录 F），我们检查了以下超参数：

- 学习率： $\{4e-2, 2e-2, 1e-2, 5e-3, 3e-3\}$ ，选择： $2e-2$ 、
- 训练步骤中的字典标记数： $\{16K, 32K\}$ ，选择：32K。需要注意的是，在训练过程中，专门用于字典的文档标记数与专门用于问题的标记数相同。

对于其他大多数超参数的选择，我们遵循了[Wu 等人，2022]，以提供公平的比较。

E.1 d

在第 5.3、5.4 和 5.5 节中，对于 $d \in \{1, 2, 4, 8\}$ 的模型，我们使用了恒定计划。对于 $d=64$ 的模型，我们在 450k 步的训练中使用了 $d=2$ ，并在最后 50k 步切换到 $d=64$ 。

64。在附录 F 中，我们使用 $d=1$ 进行训练，直到模型的准确率达到 98%，然后改用 $d=128$ 。对于第 5.2 节中的 184M 模型我们在每个训练步骤中随机从 $\{2, 128\}$ 中抽取 d 。

F 词典查找任务

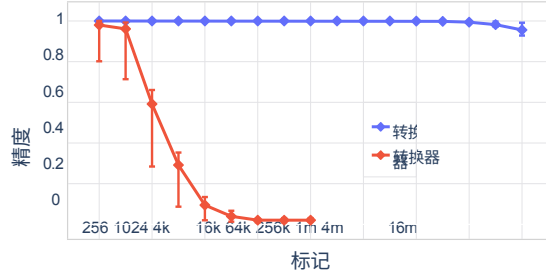


图 10：查字典任务中准确率与字典标记数的关系。任务格式如下

$\langle k \rangle k_1 \langle v \rangle v_1 \langle k \rangle k_2 \langle v \rangle v_2 \dots \langle k \rangle k_n \langle v \rangle v_n \langle q \rangle q_1 \langle v \rangle v_1 \dots$ where is provided a dictionary, followed by queries on randomly selected keys. 准确度通过测量 $\langle q \rangle$ 符号后的预测值 v_i 来确定。模型在包含 512 个词组的示例中进行了训练，并使用扩展的上下文长度进行了评估。即使内存容量很大，FOT 也能表现出很高的准确性。基线转换器在处理 16K 词组时，基线转换器已经失效。误差条代表 10 个种子的最小值和最大值。

我们提出了一项字典查询任务，以测试使用交叉批处理方法训练的模型能否利用大容量内存数据库。该任务中的文档分为两部分。第一部分使用格式记录定义键及其相关值：

$$\langle k \rangle, k_1, k_2, k_3, k_4, \langle v \rangle, v_1, v_2, v_3, v_4,$$

其中 $\langle k \rangle$ 是一个特殊标记，表示定义序列的开始、

第二部分是查询与之前定义的键相关的值。查询格式如下

$$\langle q \rangle, k_1, k_2, k_3, k_4, \langle v \rangle, v_1, v_2, v_3, v_4,$$

其中 $\langle q \rangle$ 是表示查询开始的特殊标记。我们掩盖了损失，因此对于这样一个问题，只包括 v_1, v_2, v_3, v_4 。我们使用 64 个标记的词汇，键和值用 4 个标记来描述。

在训练过程中，我们使用由 512 个标记组成的文档。每份文档的前半部分是定义，后半部分是问题。对于 FOT，我们使用 256 个局部上下文，因此模型需要使用记忆注意力层来正确回答问题。我们从 $d=1$ 开始，一旦模型能够达到 98% 的训练准确率，就将其增加到 $d=128$ 。在推理过程中，我们使用 $k=32$ （kNN 检索的键数）。作为基线，我们使用一个以上下文长度为 512 进行训练的标准转换器模型。在评估过程中，我们测试了不同的局部上下文长度，结果很快就很糟糕。

在评估中，我们使用较长的文档，但只将最后 256 个标记与问题相对应。也就是说，随着上下文变大（图 10），中的标记轴定义的数量会增加，但问题的数量保持不变。

G FOT 微调

为了在表 3 中进行比较，我们对模型进行了 100k 步的预训练，总批次大小为 128（每步 128K 标记，本地上下文为 1024）。然后，我们以相同的批量大小对 FOT 和基线进行了额外的 10k 步微调。在对 FOT 进行微调时，我们在每次训练中随机从

$\{2, 128\}$ 中随机取样，以防止模型在训练过程中过度适应较大的额外上下文长度。

H 数据集

第 5.1 节概述了本研究中使用的 PG-19 和 arXiv 数据集的基本细节。现在，我们将介绍其余数据集的详细信息：

GitHub 我们使用 BigQuery 获取了大量获得许可的 Github 代码库。通过过滤特定的文件扩展名（C、C++、Java、Python、Go 和 TypeScript），我们捕获了单个源代码文件，这些文件通常很短，但在源代码库中有大量的依赖关系和交叉引用。为了保留结构，同时以随机的顺序对文件和子目录进行洗牌，我们对每个资源库中的所有文件进行了串联，将子目录视为一个单元，这与 Wu 等人的做法类似[2022]。

伊莎贝尔 (Isabelle) 伊莎贝尔语料库包括以形式语言编写的理论形式出现的形式数学证明。我们将《形式化证明档案》（自 2021 年 10 月起）³和 Isabelle 标准库中的理论结合起来，创建了一个以开源方式授权的理论语料库。每个理论都侧重于基础逻辑、高级分析、代数或密码学等主题，由多个包含证明的文件组成。与 GitHub 语料库类似，每个理论中的文件都被串联成一个文档。不过，与 Github 语料库不同的是，我们根据文件的导入依赖关系来排列文件，确保后面的文件可以利用前面文件中证明的子定理。

I 硬件和技术细节

我们使用了谷歌云平台（GCP）的 TPU 虚拟机。每台 TPU 虚拟机都有 8 个 TPUv2 / TPUv3 内核，共计 64GB / 128GB 的设备内存、96 个 CPU 内核和超过 300GB 的内存。在更大规模的实验中（第 5.2 节），我们使用了拥有 32 个 TPUv3 内核的机器。为了训练 LONGLLAMA 检查点，我们使用了 TPU Research Cloud 提供的 TPUv3-128 pod，在此表示感谢。

J 随机性

为了评估结果的显著性，我们对研究中的部分实验进行了多次运行。在图 10，中我们计算了误差条，显示了同一实验 10 次运行的最小值和最大值。对于附录 K，中的 arXiv 基线实验我们使用不同的随机种子进行了三次运行，并计算了它们的标准偏差，标准偏差等于 0.002 perplexity。然而，由于资源限制，我们无法对所有实验进行多次运行。我们的初步研究结果表明，与调查中观察到的其他因素的影响相比，观察到的差异很小。

在计算测试复杂度时，我们使用了 100 万个标记。

K 其他实验结果

本节将介绍更多实证结果，详细比较 FOT 与 Memorizing Transformer（吴等人，2022 年）基线。这两个模型都是按照[Wu 等，人2022]。的方法，在单文档设置下，以相同的 500k 步数和 2K 的局部上下文进行训练，并在 arXiv 数据集上进行评估我们特别研究了以给定上下文长度训练的模型在以不同上下文长度进行评估时的表现。这些实验与第 5.2，节中的实验不同因为模型是在同一个数据集（arXiv）上训练和评估的，而不像第 5.2 节中的 C4 训练和零次评估。

表 10 中内存长度为 2K 的 MT 基线很难有效利用 32K 词组以外的额外上下文。当使用更长的上下文进行评估时，使用 8K 内存训练的模型表现明显更好，在 64K 字节时显示出更多的可解性提升。当训练内存长度增加到 16K 及以上时，我们观察到收益递减。

使用相同的设置，我们研究了 FOT 在改变 d 和 w 配置时的性能，与第 5.4 节类似，见表 11。参数值 $w=1$ 和 $w=2$ 对应于额外的 d 和 w 配置。

³ <https://www.isa-afp.org>

上下文长度分别为 2K 和 4K。在与 MT 的 2K 附加上下文长度的对比中，FOT 的表现优于 MT 基线，这说明了可训练密钥和值的重要性（另见 C.1）。此外，我们还证实了第 5.4 节中的结论，即在所有情况下， $d=2$ 都明显优于 $d=1$ 。我们的最佳配置在训练过程中使用 4K 附加上下文时可达 2.148 的困惑度，而使用 16K 附加上下文时 MT 的困惑度为 2.164。

表 10：记忆变换器：不同训练内存长度和评估上下文大小的 arXiv 难解性值

	评估上下文	训练内存长度			
		2k	8k	16k	32k
	4K	2.309	2.334	2.348	2.365
	8K	2.242	2.244	2.252	2.265
	16K	2.215	2.206	2.206	2.215
	32K	2.199	2.178	2.177	2.181
	64K	2.195	2.169	2.166	2.168
	128K	2.195	2.168	2.164	2.166

表 11：FoT：不同参数组合和评估上下文大小的 arXiv 易错性值

	评估上下文	参数组合 (w, d)、训练内存长度			
		(1, 1), 2048	(1, 2), 2048	(2, 1), 4096	(2, 2), 4096
	4K	2.292	2.299	2.305	2.309
	8K	2.229	2.224	2.214	2.217
	16K	2.206	2.194	2.178	2.178
	32K	2.192	2.176	2.159	2.156
	64K	2.187	2.171	2.152	2.149
	128K	2.187	2.171	2.152	2.148

L 代码

在清单 2 中，我们展示了 FOTs 的注意力代码（即记忆注意力层和交叉批处理训练的代码），参见第 3 节附录 B.2、附录 B.3。我们注意到，对代码的改动很小；这些改动仅限于记忆层（其他层遵循标准转换器协议），不需要任何新的可训练参数。

清单 2：记忆注意力：让 ℓ 成为记忆注意力层。在训练过程中，我们让 ℓ 关注本地上下文、上一个本地上下文和来自其他文档的 $d-1$ 上下文中的（键、值）对。在推理过程中， ℓ 的查询会关注本地上下文和从内存中获取的 k 个近邻。为简单起见，我们提供了一个头部的代码，并假设交叉批次维度 d 等于批次大小。

```
def mem_attn_layer ( Ql , Kl , Vl , Cl , Km , Vm , Kp , Vp , attn_scf , mode ) : """用于交叉
批处理和记忆注意的注意机制
参数：
    Ql , Kl , Vl : 形状为 [ batch , ctx_len , dim ] 的张量。
                    Km , Vm : 形状 [ batch , ctx_len , k , dim ] 的张量
                        与      的 k 个最匹配的内存键
                    Ql 中的每个查询以及相关值 Kp , Vp : 形状为 [ batch , ctx_len ,
dim ] 的张量，带有
                        的键和值的张量： 在 softmax 之前使用的比例因子
    模式：          训练或推理 返回值
    y : 具有形状 [ batch , ctx_len , dim ] 的向量 """
# 关注本地上下文
local_attention= jnp . einsum ( " bqd , bkd ->bqk " , Ql , Kl )
local_attention *= attn_scf
local_attention= apply_causal_mask ( local_attention )

if mode== " train " :
    # prev_attention= jnp . einsum ( " bqd , ckd -> bqck " , Ql , Kp )
    shape= prev_attention . shape
    additional_attention= prev_attention . reshape ( shape [: -2]+ ( -1 , ) ) elif mode==
" inference " :
    # 在推理模式中，我们额外使用了从内存中获取的最近邻 #。我们为每个查询检索 k ( key ,
value )
    每个查询的 # 对。
    memory_attention= jnp . einsum ( " bqd , bqnd -> bqk " , Ql , Km ) additional_attention=
memory_attention
else :
    raise Exception ( " 只支持训练和推理模式 " )

additional_attention *= attn_scf

# 我们合并原始注意力分数并计算软最大值 combined_attention= jnp . concatenate ( [
local_attention ,
                                additional_attention ], 轴
                                = -1)
combined_weights= jax . nn . softmax ( combined_attention , axis= -1)

ctx_len= Ql . shape [1]
local_weights= combined_weights [ ... , : ctx_len ]
additional_weights= combined_weights [ ... , ctx_len : ]

y= jnp . einsum ( " bqk , bkd -> bqk " , local_weights , Vl )

if mode== " train " :
    prev_weights= additional_weights
    shape= prev_weights . shape
    prev_weights= prev_weights . reshape ( shape [: -1]+ ( -1 , ctx_len ) ) y+= jnp .
einsum ( " bqck , ckd -> bqk " , prev_weights , Vp )
else :
    memory_weights= additional_weights
    y+= jnp . einsum ( " bqk , bqnd -> bqk " , memory_weights , Vm )
return y
```